

RaceDay

Part 1: System Planning and Database

Student Number: St10470874

Full Name: Mihle Mambikimba

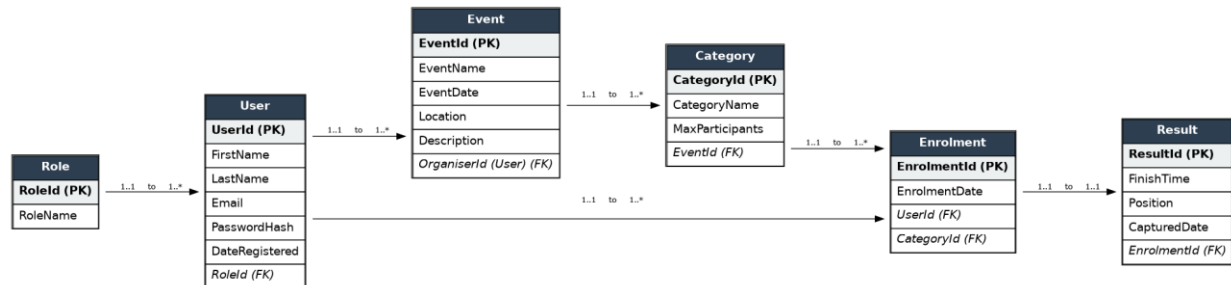
Module Code: PROG6212

Due Date: 4 September 2026

GitHub Repository Link:

Section A –

The diagram below shows the full data model for the RaceDay system: six entities, their attributes, primary keys (PK), foreign keys (FK), and the cardinality of every relationship.



Relationship Summary

- Role → User (1 to Many): one Role applies to many Users; each User has exactly one Role.
- User → Event (1 to Many): one Organiser (a User) can create many Events; each Event has exactly one owning Organiser.
- Event → Category (1 to Many): one Event can have many Categories; each Category belongs to exactly one Event.
- User → Enrolment (1 to Many): one Participant (a User) can have many Enrolments; each Enrolment belongs to exactly one Participant.
- Category → Enrolment (1 to Many): one Category can have many Enrolments; each Enrolment is for exactly one Category.
- Enrolment → Result (1 to 1): each Enrolment produces exactly one Result once captured.

The Participant-to-Category relationship is conceptually many-to-many (a Participant can enter many Categories; a Category has many Participants). This is resolved through the Enrolment junction entity, which converts the many-to-many relationship into two one-to-many relationships.

Section B

The table below lists every API endpoint the RaceDay system will expose, covering Authentication, User Profile, Events, Categories, Event Enrolments, and Results. All routes begin with /api/.

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
Authentication					
POST	/api/auth/register	Creates a new User account with a specified Role	None (public)	{ firstName, lastName, email, password, role }	201 Created – user record; 400 Bad Request – invalid/missing fields; 409 Conflict – email already registered
POST	/api/auth/login	Authenticates a User and returns a token	None (public)	{ email, password }	200 OK – auth token + user info; 401 Unauthorized – invalid credentials
User Profile					
GET	/api/users/{id}	Retrieves a specific user's profile	Any (logged in)	None	200 OK – user profile; 404 Not Found – user does not exist
PUT	/api/users/{id}	Updates the logged-in user's own profile details	Any (own profile)	{ firstName, lastName, email }	200 OK – updated profile; 400 Bad Request – invalid data; 404 Not Found
Events					
GET	/api/events	Retrieves a list of all events (for browsing)	None (public)	None	200 OK – list of events
GET	/api/events/{id}	Retrieves details for a single event	None (public)	None	200 OK – event details; 404 Not Found
POST	/api/events	Creates a new event	Organiser	{ eventName, eventDate, location, description }	201 Created – new event; 400 Bad Request – invalid/missing fields
PUT	/api/events/{id}	Updates an existing event's details	Organiser	{ eventName, eventDate, location, description }	200 OK – updated event; 400 Bad Request; 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
DELETE	/api/events/{id}	Deletes an event	Organiser	None	200 OK – confirmation; 404 Not Found
Categories					
GET	/api/events/{eventId}/categories	Retrieves all categories for a specific event	None (public)	None	200 OK – list of categories; 404 Not Found – event does not exist
POST	/api/events/{eventId}/categories	Adds a new category to an event	Organiser	{ categoryName, maxParticipants }	201 Created – new category; 400 Bad Request; 404 Not Found – event does not exist
PUT	/api/categories/{id}	Updates a category	Organiser	{ categoryName, maxParticipants }	200 OK – updated category; 404 Not Found
DELETE	/api/categories/{id}	Deletes a category	Organiser	None	200 OK – confirmation; 404 Not Found
Event Enrolments					
POST	/api/categories/{categoryId}/enrolments	Enrols the logged-in Participant into a category	Participant	None	201 Created – enrolment record; 404 Not Found – category doesn't exist; 409 Conflict – already enrolled / category full
GET	/api/users/{userId}/enrolments	Retrieves all enrolments for a specific Participant	Participant (own) / Organiser	None	200 OK – list of enrolments; 404 Not Found
GET	/api/events/{eventId}/enrolments	Retrieves all enrolments for an event (across all categories)	Organiser	None	200 OK – list of enrolments; 404 Not Found
DELETE	/api/enrolments/{id}	Cancels/withdraws an enrolment	Participant (own)	None	200 OK – confirmation; 404 Not Found
Results					
POST	/api/enrolments/{enrolmentId}/result	Captures a result for a specific enrolment	Organiser	{ finishTime, position }	201 Created – result record; 404 Not Found – enrolment doesn't exist; 409 Conflict –

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					result already captured
GET	/api/users/{userId}/results	Retrieves a Participant's own results (personal history)	Participant (own) / Organiser	None	200 OK – list of results; 404 Not Found
GET	/api/events/{eventId}/results	Retrieves all results for an event	None (public)	None	200 OK – list of results; 404 Not Found

Section C

I wrote the script below and executed it in SQL Server Management Studio, and I created the tables in dependency order (Role → User → Event → Category → Enrolment → Result) so that foreign key references always point to an existing table.

Table

```
1. Role
CREATE TABLE Role (
  RoleId INT IDENTITY(1,1) PRIMARY KEY,
  RoleName VARCHAR(50) NOT NULL UNIQUE
);

-- 2. User
CREATE TABLE [User] (
  UserId INT IDENTITY(1,1) PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50) NOT NULL,
  Email VARCHAR(100) NOT NULL UNIQUE,
  PasswordHash VARCHAR(255) NOT NULL,
  RoleId INT NOT NULL,
  DateRegistered DATETIME NOT NULL DEFAULT GETDATE(),
  CONSTRAINT FK_User_Role FOREIGN KEY (RoleId) REFERENCES Role(RoleId)
);

-- 3. Event
CREATE TABLE Event (
  EventId INT IDENTITY(1,1) PRIMARY KEY,
  EventName VARCHAR(100) NOT NULL,
  EventDate DATE NOT NULL,
  Location VARCHAR(100) NOT NULL,
  Description VARCHAR(500) NULL,
  OrganiserId INT NOT NULL,
  CONSTRAINT FK_Event_User FOREIGN KEY (OrganiserId) REFERENCES [User](UserId)
);

-- 4. Category
CREATE TABLE Category (
  CategoryId INT IDENTITY(1,1) PRIMARY KEY,
  EventId INT NOT NULL,
  CategoryName VARCHAR(50) NOT NULL,
  MaxParticipants INT NOT NULL DEFAULT 100,
  CONSTRAINT FK_Category_Event FOREIGN KEY (EventId) REFERENCES Event(EventId)
);

-- 5. Enrolment
CREATE TABLE Enrolment (
  EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
  UserId INT NOT NULL,
  CategoryId INT NOT NULL,
  EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),
  CONSTRAINT FK_Enrolment_User FOREIGN KEY (UserId) REFERENCES [User](UserId),
  CONSTRAINT FK_Enrolment_Category FOREIGN KEY (CategoryId) REFERENCES Category(CategoryId),
  CONSTRAINT UQ_Enrolment_UserCategory UNIQUE (UserId, CategoryId)
);

-- 6. Result
CREATE TABLE Result (
```

```

ResultId INT IDENTITY(1,1) PRIMARY KEY,
EnrolmentId INT NOT NULL UNIQUE,
FinishTime TIME NOT NULL,
Position INT NULL,
CapturedDate DATETIME NOT NULL DEFAULT GETDATE(),
CONSTRAINT FK_Result_Enrolment FOREIGN KEY (EnrolmentId) REFERENCES Enrolment(EnrolmentId)
);

```

```

-- Roles
INSERT INTO Role (RoleName) VALUES ('Organiser'), ('Participant');

-- Users (2 Organisers, 2 Participants)
INSERT INTO [User] (FirstName, LastName, Email, PasswordHash, RoleId) VALUES
('Thabo', 'Rakhale', 'thabo.rakhale@raceday.co.za', 'hashedpassw', 1),
('Ntsakoe', 'Sefe', 'ntsako.sefele@raceday.co.za', 'hashedpassword', 1),
('Aseza', 'Makanda', 'aseza.makadna@raceday.co.za', 'hashedpword', 2),
('Leshka', 'Pitso', 'leshka.pitso@raceday.co.za', 'hashed_pw', 2);

-- Events (3 events, owned by the two Organisers)
INSERT INTO Event (EventName, EventDate, Location, Description, OrganiserId) VALUES
('Sprinting', '2022-11-28', 'eTekwini', 'Usain Bolts competitor at uShaka Marine.', 1),
('Marathon', '2026-11-30', 'Mthatha', 'Community road running event through Mthatha.', 1),
('Park Run Charity Walk', '2026-05-16', 'Gleharvie', 'Charity walk supporting local community projects.', 2);

-- Categories (per event)
INSERT INTO Category (EventId, CategoryName, MaxParticipants) VALUES
(1, '100m Sprint', 500),
(1, '34km Marathon', 300),
(2, '12km Marathon', 400),
(2, '41km Half Marathon', 400),
(3, '10km Walk', 200);

-- Sample Enrolments
INSERT INTO Enrolment (UserId, CategoryId) VALUES
(3, 1), -- Aseza enters the 100 Sprint
(4, 3), -- Ntsako enters the 34km Marathon
(3, 5); -- Leshka enters the 10km Walk

-- Sample Results
INSERT INTO Result (EnrolmentId, FinishTime, Position) VALUES
(1, '04:32:10', 45),
(2, '03:58:22', 12);

```

Insert screenshot of the INSERT statements executing successfully in SSMS here.

C.3 – Verification Queries

The following queries were run to confirm the data was inserted correctly.

```

SELECT * FROM Role;
SELECT * FROM [User];
SELECT * FROM Event;
SELECT * FROM Category;
SELECT * FROM Enrolment;
SELECT * FROM Result;

```

Visual Studio Code interface showing a SQL query execution in the 'RaceDays.sql' file. The query is executed successfully, and the results are displayed in the 'Results' pane.

```
133 1,4, 00.00.00.00, 1.4,7,
134
135
136 -- In this section we verify queries to confirm whether data
137 -- was put correctly
138
139
140 SELECT * FROM Role;
141 SELECT * FROM [User];
142 SELECT * FROM Event;
143 SELECT * FROM Category;
144 SELECT * FROM Enrolment;
145 SELECT * FROM Result;
146
```

Results:

RoleId	RoleName
1	Organiser
2	Participant

Query executed successfully. (localdb)\api_guide (15.0 RTM) | LabVM2049939:Student (52) | master | 00:00:00 | Row: 1, Col: 1 | 2 rows

Visual Studio Code interface showing a SQL query execution in the 'RaceDays.sql' file. The query is executed successfully, and the results are displayed in the 'Results' pane.

```
133 1,4, 00.00.00.00, 1.4,7,
134
135
136 -- In this section we verify queries to confirm whether data
137 -- was put correctly
138
139
140 SELECT * FROM Role;
141 SELECT * FROM [User];
142 SELECT * FROM Event;
143 SELECT * FROM Category;
144 SELECT * FROM Enrolment;
145 SELECT * FROM Result;
146
```

Results:

UserId	FirstName	LastName	Email	PasswordHash	RoleId	DateRegistered
1	Thabo	Rakhale	thabo.rakhale@raceday.com	hashedpassw	1	2025-09-04 19:34:09.247
2	Ntsako	Sefe	ntsako.sefe@raceday.com	hashedpassword	1	2025-09-04 19:34:09.247
3	Aseza	Makanda	aseza.makanda@raceday.com	hashedpw	2	2025-09-04 19:34:09.247
4	Leshika	Pitso	leshika.pitso@raceday.com	hashed_pw	2	2025-09-04 19:34:09.247

Query executed successfully. (localdb)\api_guide (15.0 RTM) | LabVM2049939:Student (52) | master | 00:00:00 | Row: 1, Col: 1 | 4 rows